

Student Management React App - Documentation

File Structure Overview

src/

??? App.js	# Defines all the routes
??? create.js	# Create new student
??? edit.js	# Edit existing student
??? view.js	# View student details
??? StudentTable.js	# Main listing page with all students
??? index.css	# Global CSS styling

App.js ? Routing Entry Point

- `BrowserRouter``: Handles client-side routing using URL hash/history.
- `Routes``: Wrapper for all `Route`` definitions.
- `Route``: Maps a path to a component (like `/student/edit/:id` ? `Edit.js``).

create.js ? Add New Student

- `useState``: Stores form fields (id, name, phone, place).
- `useNavigate``: Redirects to another route after submission.
- `handleSubmit``: Sends a POST request to the backend.

edit.js ? Update Existing Student

- `useParams``: Reads studentId from the route.
- `useEffect``: Pre-fills fields by fetching data from backend.
- `handleSubmit``: Sends a PUT request to update data.

Student Management React App - Documentation

view.js ? Display Student Details

- `useParams`: Extracts ID from URL.
- `useEffect`: Fetches student details on page load.

StudentTable.js ? Main Dashboard

- Displays all students using a table.
- Buttons to View, Edit, Delete each record.
- `handleDelete`: Sends DELETE request and refreshes list.

index.css ? Styling

- Responsive design using Flexbox.
- Button-specific styling: `.btn-add`, `.btn-save`, `.btn-view`, `.btn-edit`, `.btn-delete`.
- Table styling: hover effects, borders, padding.

Backend Setup (JSON Server)

- Install: `npm install -g json-server`
- `db.json`: Contains initial student data.
- Start: `json-server --watch db.json --port 8000`

Student Management React App - Documentation

HTTP Methods Used

Action	HTTP Method	Endpoint	
-----	-----	-----	
List	GET	/students	
View One	GET	/students/:id	
Create	POST	/students	
Update	PUT	/students/:id	
Delete	DELETE	/students/:id	

Technologies Used

- React: Component-based UI.
- React Router: Routing system.
- JSON Server: REST mock API.
- fetch API: To make API calls.
- useState/useEffect/useParams/useNavigate: React hooks.

Best Practices Followed

- Component separation.
- Form validation.
- RESTful structure.
- UI/UX improvements.
- Button color separation for clarity.